МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «Программная инженерия»

# Лабораторная работа № 2

по дисциплине «Математические Основы Баз Данных»

по теме: «Базовые SQL-запросы и фильтрация данных»

Выполнил студент группы:
БВТ2505
Коновалова Варвара Вячеславовна

Проверил:
Быватов Семён Юрьевич

Москва 2026

**Оглавление**

Введение ..... 3
Выполнение ..... 4
Заключение ..... 20

## Введение

### Цель и задачи работы:

Целью данной лабораторной работы является приобретение практических навыков наполнения реляционной базы данных тестовыми данными и составления базовых SQL-запросов на выборку и фильтрацию в СУБД PostgreSQL, а также закрепление знаний о структуре команды SELECT, предикатах отбора строк, трёхзначной логике и работе с NULL.

### Задачи:

1. Наполнить все таблицы базы данных «Онлайн-зоомагазин» согласованными данными, используя generate_series и random(); в крупные таблицы (товары, заказы) внести не менее тысячи строк.
2. Составить запрос, выводящий заданные столбцы товаров без дубликатов (DISTINCT) с псевдонимами столбцов.
3. Составить запрос с фильтрацией по нескольким условиям, объединённым AND и OR со скобками.
4. Применить предикаты BETWEEN, IN, LIKE (ILIKE) и IS NULL - каждый хотя бы один раз.
5. Составить запрос с сортировкой по двум столбцам в разных направлениях.
6. Реализовать постраничный вывод товаров с помощью LIMIT и OFFSET.
7. Для каждого запроса привести текст, назначение и фрагмент результата.

**Выполнение**

1. Наполните все таблицы согласованными данными, используя generate_series или иной генератор для массового заполнения.

## 1. Наполнение таблиц данными

### 1.1. Принципы наполнения

Наполнение выполняется командой INSERT. Для массовой генерации строк используется функция generate_series(1, N), которая возвращает последовательность чисел от 1 до N; на каждое число создаётся одна строка. Для разнообразия значений применяются функция random() (случайное число от 0 до 1) и выбор элемента массива по индексу, например (ARRAY['dog','cat','bird'])[1 + g % 3].

Чтобы данные были согласованными, соблюдены следующие правила:

- таблицы заполняются в порядке зависимостей по внешним ключам: сначала справочники (brands, categories) и users, затем products, stock, carts, orders и только после них - позиции, отзывы, адреса и доставки;
- внешние ключи генерируются только в диапазоне существующих id (например, user_id - от 1 до 300, category_id - от 6 до 12, то есть только подкатегории);
- перед наполнением выполняется TRUNCATE ... RESTART IDENTITY CASCADE: скрипт можно запускать повторно, а идентификаторы всегда начинаются с 1;
- вызов setseed(0.42) фиксирует генератор случайных чисел, поэтому при повторном запуске получаются те же данные;
- цена в order_items копируется из products.price, а итог заказа orders.total пересчитывается по его позициям;
- часть значений намеренно оставлена NULL (телефон, описание товара, порода, квартира), а 10% товаров скрыты (is_active = FALSE) - это нужно для запросов с IS NULL и фильтрацией.

### 1.2. Плановый объём данных

| Таблица | Строк | Способ генерации |
|---|---|---|
| brands | 10 | явный список VALUES |
| categories | 12 | 5 корневых + 7 подкатегорий (иерархия по parent_id) |
| users | 300 | generate_series(1, 300) |
| pets | 400 | generate_series(1, 400), владелец - случайный |
| products | 1000 | generate_series(1, 1000) |
| stock | 1000 | INSERT ... SELECT из products |
| carts | 150 | generate_series(1, 150) |
| cart_items | 450 | по 3 товара в каждой корзине |
| orders | 1200 | generate_series(1, 1200) |
| order_items | 2400 | от 1 до 3 позиций на заказ |
| reviews | 600 | уникальные пары (пользователь, товар) |
| addresses | 1200 | по одному адресу на заказ |
| deliveries | 1200 | по одной доставке на заказ |

### 1.3. Подготовка: фиксация случайности и очистка таблиц

```
-- Фиксируем генератор случайных чисел, чтобы данные были
воспроизводимыми
SELECT setseed(0.42);

-- Очистка всех таблиц и сброс счётчиков id (повторный запуск)
TRUNCATE TABLE deliveries, addresses, reviews, order_items, orders,
              cart_items, carts, stock, products, brands, categories,
              pets, users
RESTART IDENTITY CASCADE;
```

### 1.4. Бренды и категории

Бренды вносятся списком значений. Категории - в два приёма: сначала корневые (parent_id = NULL), затем подкатегории, у которых parent_id определяется подзапросом по slug родителя. Так реализуется иерархия из первой работы. Подкатегории получают id 6–12.

```
-- 1.4 Бренды (10 строк)
INSERT INTO brands (name, country) VALUES
   ('Royal Canin', 'Франция'),
   ('Pro Plan',    'Франция'),
```

```
    ('Trixie',      'Германия'),
    ('Beaphar',     'Нидерланды'),
    ('Hill''s',     'США'),
    ('Pedigree',    'США'),
    ('Whiskas',     'США'),
    ('Ferplast',    'Италия'),
    ('Hartz',       'США'),
    ('Четвероногий друг', 'Россия');

-- 1.4 Категории: 5 корневых (parent_id = NULL) + 7 подкатегорий
INSERT INTO categories (parent_id, name, slug) VALUES
    (NULL, 'Корма',             'food'),
    (NULL, 'Игрушки',           'toys'),
    (NULL, 'Аксессуары',        'accessories'),
    (NULL, 'Гигиена и уход',    'care'),
    (NULL, 'Лежанки и домики',  'beds');

INSERT INTO categories (parent_id, name, slug) VALUES
    ((SELECT id FROM categories WHERE slug = 'food'),        'Сухой корм',
'dry-food'),
    ((SELECT id FROM categories WHERE slug = 'food'),        'Влажный корм',
'wet-food'),
    ((SELECT id FROM categories WHERE slug = 'toys'),        'Мячи и фрисби',
'balls'),
    ((SELECT id FROM categories WHERE slug = 'toys'),        'Когтеточки',
'scratchers'),
    ((SELECT id FROM categories WHERE slug = 'accessories'), 'Ошейники и
поводки',  'collars'),
    ((SELECT id FROM categories WHERE slug = 'care'),        'Шампуни и
косметика', 'shampoo'),
    ((SELECT id FROM categories WHERE slug = 'beds'),        'Лежанки',
'sleeping-beds');
-- Подкатегории получают id 6..12 (по порядку вставки)
```

### 1.5. Пользователи

Email уникален за счёт номера g. У каждого десятого пользователя телефон не указан (NULL). Имя и фамилия выбираются из массивов с учётом рода, дата регистрации - случайная за последний год.

```
-- 1.5 Пользователи (300 строк)
INSERT INTO users (email, phone, first_name, last_name, created_at)
SELECT 'user' || g || '@example.com',
```

```
    CASE WHEN g % 10 = 0 THEN NULL                    -- у 10% телефон не
указан
        ELSE '+7900' || lpad((1000000 + g)::text, 7, '0') END,
    CASE WHEN g % 2 = 0
        THEN (ARRAY['Иван','Пётр','Алексей','Дмитрий','Сергей'])[1 + (g /
2) % 5]
        ELSE (ARRAY['Ольга','Мария','Анна','Елена','Наталья'])[1 + (g / 2) %
5] END,
    CASE WHEN g % 2 = 0
        THEN (ARRAY['Петров','Иванов','Смирнов','Кузнецов','Попов'])[1 +
(g / 3) % 5]
        ELSE
(ARRAY['Петрова','Иванова','Смирнова','Кузнецова','Попова'])[1 + (g / 3) %
5] END,
    now() - random() * 365 * interval '1 day'
FROM generate_series(1, 300) AS g;
```

### 1.6. Питомцы

Владелец выбирается случайно из 300 пользователей. Порода зависит от вида животного, у каждого пятого питомца она не указана.

```
-- 1.6 Питомцы (400 строк, владелец выбирается случайно)
INSERT INTO pets (user_id, name, species, breed, birth_date, notes)
SELECT 1 + floor(random() * 300)::int,

(ARRAY['Барсик','Шарик','Мурка','Рекс','Кеша','Луна','Жучка','Симба'])[1 +
g % 8],
    (ARRAY['dog','cat','bird'])[1 + g % 3],
    CASE WHEN g % 5 = 0 THEN NULL                    -- порода
неизвестна
        ELSE CASE g % 3
            WHEN 0 THEN (ARRAY['Лабрадор','Такса','Хаски'])[1 + (g / 3)
% 3]
            WHEN 1 THEN (ARRAY['Британская','Сфинкс','Мейн-кун'])[1 +
(g / 3) % 3]
            ELSE       (ARRAY['Волнистый
попугай','Канарейка','Корелла'])[1 + (g / 3) % 3]
        END
    END,
    current_date - (random() * 3650)::int,
    CASE WHEN g % 7 = 0 THEN 'Аллергия на курицу' END
FROM generate_series(1, 400) AS g;
```

### 1.7. Товары

Основная таблица для запросов. Название строится из типа товара, серии и номера (например, «Сухой корм «Premium» №15»). Цена - случайная от 100 до 5000, описание отсутствует у каждого четвёртого товара, каждый десятый товар скрыт.

```
-- 1.7 Товары (1000 строк)
INSERT INTO products (brand_id, category_id, title, description, price,
is_active, created_at)
SELECT s.brand_id,
      s.category_id,
      (ARRAY['Сухой корм','Влажный
корм','Мяч','Когтеточка','Ошейник','Шампунь','Лежанка'])[s.category_id - 5]
        || ' «' || (ARRAY['Premium','Classic','Light','Sensitive','Active'])[1 + s.g %
5]
        || '» №' || s.g,
      CASE WHEN s.g % 4 = 0 THEN NULL                    -- у 25% нет
описания
           ELSE 'Описание товара №' || s.g END,
      round((random() * 4900 + 100)::numeric, 2),          -- цена от 100 до 5000
      (s.g % 10 <> 0),                                   -- 10% товаров скрыты
      now() - random() * 730 * interval '1 day'
FROM (SELECT g,
           1 + floor(random() * 10)::int AS brand_id,
           6 + floor(random() * 7)::int  AS category_id
     FROM generate_series(1, 1000) AS g) AS s;
```

### 1.8. Складские остатки

Для каждого товара создаётся ровно одна запись (связь 1:1). Часть товаров имеет нулевой остаток.

```
-- 1.8 Складские остатки (по строке на каждый товар = 1000 строк)
INSERT INTO stock (product_id, quantity)
SELECT id,
      CASE WHEN id % 7 = 0 THEN 0                        -- часть товаров
закончилась
           ELSE floor(random() * 200)::int END
FROM products;
```

### 1.9. Корзины и позиции корзин

Каждая корзина содержит три разных товара; формула ((c.id * 7 + k * 13) % 1000) + 1 гарантирует отсутствие повторов и не нарушает составной первичный ключ.

```
-- 1.9 Корзины (150) и позиции корзин (по 3 товара)
INSERT INTO carts (user_id, status, created_at)
SELECT 1 + floor(random() * 300)::int,
    (ARRAY['active','converted','abandoned'])[1 + g % 3],
    now() - random() * 60 * interval '1 day'
FROM generate_series(1, 150) AS g;

INSERT INTO cart_items (cart_id, product_id, quantity)
SELECT c.id,
    ((c.id * 7 + k * 13) % 1000) + 1,                -- разные товары в одной корзине
    1 + floor(random() * 3)::int
FROM carts AS c
CROSS JOIN generate_series(1, 3) AS k;
```

### 1.10. Заказы

Статус заказа выбирается случайно из пяти допустимых значений CHECK. Сумма total изначально равна 0 и пересчитывается после вставки позиций.

```
-- 1.10 Заказы (1200 строк); итоговая сумма пересчитывается ниже
INSERT INTO orders (user_id, status, total, created_at)
SELECT 1 + floor(random() * 300)::int,
    (ARRAY['new','paid','shipped','delivered','cancelled'])[1 + floor(random() * 5)::int],
    0,
    now() - random() * 365 * interval '1 day'
FROM generate_series(1, 1200) AS g;
```

### 1.11. Позиции заказов и пересчёт суммы

В каждом заказе от 1 до 3 различных товаров (условие k <= 1 + o.id % 3). Цена фиксируется на момент покупки. Затем UPDATE ... FROM считает сумму по позициям - так выполняется бизнес-правило «заказ содержит хотя бы одну позицию».

```
-- 1.11 Позиции заказов: от 1 до 3 различных товаров в каждом заказе
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT o.id,
       p.id,
       1 + floor(random() * 3)::int,
       p.price                                  -- цена фиксируется на момент
покупки
FROM orders AS o
CROSS JOIN generate_series(1, 3) AS k
JOIN products AS p ON p.id = ((o.id * 17 + k * 31) % 1000) + 1
WHERE k <= 1 + o.id % 3;

-- Пересчёт суммы заказа по его позициям
UPDATE orders AS o
SET total = s.t
FROM (SELECT order_id, SUM(quantity * unit_price) AS t
      FROM order_items
      GROUP BY order_id) AS s
WHERE s.order_id = o.id;
```

### 1.12. Отзывы

Пары (user_id, product_id) строятся по формулам 1 + g % 300 и 1 + (g * 7) % 1000 и не повторяются, поэтому ограничение UNIQUE (user_id, product_id) не нарушается. Оценка - случайное число от 1 до 5.

```
-- 1.12 Отзывы (600 строк, пары «пользователь–товар» не повторяются)
INSERT INTO reviews (user_id, product_id, score, comment, created_at)
SELECT 1 + g % 300,
       1 + (g * 7) % 1000,
       1 + floor(random() * 5)::int,
       CASE WHEN g % 3 = 0 THEN NULL ELSE 'Отзыв №' || g END,
       now() - random() * 300 * interval '1 day'
FROM generate_series(1, 600) AS g;
```

### 1.13. Адреса доставки

Для каждого заказа создаётся один адрес. Получатель собирается из фамилии и имени владельца заказа (JOIN с users). Квартира у каждого третьего адреса не указана.

```
-- 1.13 Адреса доставки (по одному на заказ = 1200 строк)
INSERT INTO addresses (order_id, city, street, house, apartment, postal_code,
recipient)
```

```
SELECT o.id,
    (ARRAY['Москва','Санкт-
Петербург','Казань','Екатеринбург','Новосибирск'])[1 + o.id % 5],
    (ARRAY['Ленина','Мира','Гагарина','Советская','Пушкина'])[1 + (o.id / 5)
% 5],
    (1 + o.id % 100)::text,
    CASE WHEN o.id % 3 = 0 THEN NULL ELSE (1 + o.id % 200)::text
END,
    (100000 + o.id % 899999)::text,
    u.last_name || ' ' || u.first_name
FROM orders AS o
JOIN users  AS u ON u.id = o.user_id;
```

### 1.14. Доставки

Статус доставки согласован со статусом заказа: delivered → delivered, shipped → in_transit, остальные → pending. Для самовывоза трек-номер отсутствует.

```
-- 1.14 Доставки (по одной на заказ = 1200 строк)
INSERT INTO deliveries (order_id, method, status, tracking_code, shipped_at,
delivered_at)
SELECT o.id,
    m.method,
    CASE o.status WHEN 'delivered' THEN 'delivered'
                  WHEN 'shipped'   THEN 'in_transit'
                  ELSE 'pending' END,
    CASE WHEN m.method = 'pickup' THEN NULL
        ELSE 'TRK' || lpad(o.id::text, 8, '0') END,
    CASE WHEN o.status IN ('shipped','delivered') THEN o.created_at +
interval '1 day' END,
    CASE WHEN o.status = 'delivered'            THEN o.created_at + interval '3
days' END
FROM orders AS o
CROSS JOIN LATERAL (SELECT (ARRAY['courier','pickup','post'])[1 + o.id
% 3] AS method) AS m;
```

### 1.15. Контроль количества строк

После наполнения выполняется проверочный запрос: он подсчитывает строки в каждой таблице и объединяет результаты через UNION ALL. Число строк в products и orders должно быть не меньше 1000.

```
-- 1.15 Контроль: количество строк в каждой таблице
```

```
SELECT 'users' AS table_name, COUNT(*) AS row_count FROM users
UNION ALL SELECT 'pets',        COUNT(*) FROM pets
UNION ALL SELECT 'brands',      COUNT(*) FROM brands
UNION ALL SELECT 'categories',  COUNT(*) FROM categories
UNION ALL SELECT 'products',    COUNT(*) FROM products
UNION ALL SELECT 'stock',       COUNT(*) FROM stock
UNION ALL SELECT 'carts',       COUNT(*) FROM carts
UNION ALL SELECT 'cart_items',  COUNT(*) FROM cart_items
UNION ALL SELECT 'orders',      COUNT(*) FROM orders
UNION ALL SELECT 'order_items', COUNT(*) FROM order_items
UNION ALL SELECT 'reviews',     COUNT(*) FROM reviews
UNION ALL SELECT 'addresses',   COUNT(*) FROM addresses
UNION ALL SELECT 'deliveries',  COUNT(*) FROM deliveries;
```

2. Напишите запрос, выводящий заданные столбцы товаров без дубликатов (DISTINCT), с псевдонимами столбцов.

2. **Запрос с DISTINCT и псевдонимами**

DISTINCT убирает полностью совпадающие строки результата. Псевдоним задаётся ключевым словом AS; если он содержит пробелы или кириллицу, его заключают в двойные кавычки.

3. **Запрос 1. Уникальные сочетания «бренд - категория»**

**Текст запроса:**

```
-- Запрос 1. DISTINCT + псевдонимы: уникальные сочетания «бренд – категория»
SELECT DISTINCT
    brand_id    AS "Бренд (id)",
    category_id AS "Категория (id)"
FROM products
ORDER BY brand_id, category_id;
```

**Назначение:**

Показывает, какие бренды представлены в каких категориях. В таблице 1000 товаров, но повторяющиеся пары (brand_id, category_id) выводятся один раз.

4. **Запрос 1а (дополнение). DISTINCT ON**

**Текст запроса:**

```
-- Запрос 1а (дополнение). DISTINCT ON: самый дешёвый товар в каждой
категории
SELECT DISTINCT ON (category_id)
      category_id AS "Категория (id)",
      title       AS "Название",
      price       AS "Цена"
FROM products
ORDER BY category_id, price ASC;
```

**Назначение:**

DISTINCT ON (category_id) оставляет по одной строке на каждое значение category_id. Благодаря ORDER BY category_id, price ASC из каждой категории выбирается самый дешёвый товар.

3. Напишите запрос с фильтрацией по нескольким условиям, объединённым AND и OR со скобками.

**3. Фильтрация по нескольким условиям**

Оператор AND связывает сильнее, чем OR, поэтому условия с OR обязательно заключаются в скобки. Без скобок условие a AND b OR c будет вычислено как (a AND b) OR c, что изменит смысл запроса.

**Запрос 2. Комбинация AND / OR со скобками**

**Текст запроса:**

```
-- Запрос 2. Несколько условий: AND, OR и скобки
-- Активные товары категорий 6 или 7, которые либо дешёвые, либо
дорогие
SELECT id, title, category_id, price
FROM products
WHERE is_active = TRUE
  AND (category_id = 6 OR category_id = 7)
  AND (price < 500 OR price > 4500)
ORDER BY price;
```

**Назначение:**

Выбирает активные товары из категорий 6 или 7, цена которых либо ниже 500, либо выше 4500 (самые дешёвые и самые дорогие позиции).

4. Примените в запросах предикаты BETWEEN, IN, LIKE (или ILIKE) и IS NULL - каждый хотя бы один раз.

### 4. Предикаты BETWEEN, IN, LIKE / ILIKE, IS NULL

**Запрос 3. BETWEEN**

**Текст запроса:**

```
-- Запрос 3. BETWEEN: товары в ценовом диапазоне
SELECT id, title, price
FROM products
WHERE price BETWEEN 1000 AND 1500
ORDER BY price;
```

**Назначение:**

BETWEEN 1000 AND 1500 - замкнутый диапазон, равносильный (price >= 1000 AND price <= 1500). Запрос находит товары средней ценовой категории.

**Запрос 4. IN**

**Текст запроса:**

```
-- Запрос 4. IN: товары из выбранных категорий и брендов
SELECT id, title, brand_id, category_id, price
FROM products
WHERE category_id IN (6, 7)
  AND brand_id IN (1, 2, 3)
ORDER BY id;
```

**Назначение:**

IN (...) заменяет цепочку условий через OR: товары категорий 6 и 7 от брендов с id 1, 2 и 3.

**Запрос 5. ILIKE**

**Текст запроса:**

```
-- Запрос 5. ILIKE: поиск по подстроке без учёта регистра
SELECT id, title, price
FROM products
WHERE title ILIKE '%корм%'
ORDER BY id
LIMIT 15;
```

**Назначение:**

Поиск по подстроке без учёта регистра: символ % заменяет любую последовательность символов, поэтому находятся все названия, содержащие слово «корм» («Сухой корм…», «Влажный корм…»).

*Запрос 5а (дополнение). LIKE с символами % и _*

**Текст запроса:**

```
-- Запрос 5а (дополнение). LIKE: «_» = ровно один символ, «%» = любая цепочка
-- Шампуни серии Classic с трёхзначным номером (после «№» ровно три символа)
SELECT id, title
FROM products
WHERE title LIKE 'Шампунь «Classic» №___'
ORDER BY id
LIMIT 15;
```

**Назначение:**

Символ _ заменяет ровно один символ. Шаблон '…№__' находит названия, где после знака № стоят ровно три символа, то есть шампуни серии Classic с трёхзначным номером.

**Запрос 6. IS NULL**

**Текст запроса:**

```
-- Запрос 6. IS NULL: товары без описания
SELECT id, title, description
FROM products
WHERE description IS NULL
ORDER BY id
LIMIT 15;
```

**Назначение:**

Находит товары, у которых описание не заполнено. Сравнение description = NULL всегда даёт UNKNOWN и не вернуло бы ни одной строки, поэтому применяется предикат IS NULL.

**Запрос 6а (дополнение). COALESCE**

**Текст запроса:**

```
-- Запрос 6а (дополнение). COALESCE: подстановка значения вместо
NULL
SELECT id,
      title,
      COALESCE(description, 'нет описания') AS descr
FROM products
WHERE description IS NULL
ORDER BY id
LIMIT 10;
```

**Назначение:**

Функция COALESCE возвращает первое не-NULL значение; здесь она подставляет текст «нет описания» вместо пустого значения.

**Запрос 6б (дополнение). Сравнение «= NULL» и «IS NULL»**

**Текст запроса:**

```
-- Запрос 6б (дополнение). Почему нельзя писать «= NULL» (вернёт 0
строк)
SELECT COUNT(*) AS wrong_count FROM products WHERE description =
NULL;
SELECT COUNT(*) AS right_count FROM products WHERE description IS
NULL;
```

**Назначение:**

Демонстрация трёхзначной логики: первый запрос возвращает 0, второй - реальное количество товаров без описания.

5. Напишите запрос с сортировкой по двум столбцам в разных направлениях.

**5. Сортировка по двум столбцам**
**Запрос 7. ORDER BY category_id ASC, price DESC**

**Текст запроса:**

```
-- Запрос 7. Сортировка по двум столбцам в разных направлениях
-- Категория по возрастанию, внутри категории – от дорогих к дешёвым
SELECT id, title, category_id, price
FROM products
WHERE is_active
```

```
ORDER BY category_id ASC, price DESC
LIMIT 15;
```

**Назначение:**

Товары группируются по возрастанию номера категории, а внутри каждой категории идут от самых дорогих к самым дешёвым. Второй ключ применяется только к строкам с одинаковым первым.

6. Реализуйте постраничный вывод товаров с помощью LIMIT и OFFSET.

**6. Постраничный вывод**

LIMIT задаёт размер страницы, OFFSET - число пропускаемых строк: OFFSET = (номер_страницы - 1) × размер_страницы. Для воспроизводимого результата порядок должен быть однозначным, поэтому к сортировке по цене добавлен второй ключ id: при равных ценах порядок строк не «плавает» между страницами.

**Запрос 8. Страница 1 (OFFSET 0)**

**Текст запроса:**

```
-- Страница 1
SELECT id, title, price
FROM products
WHERE is_active
ORDER BY price DESC, id ASC
LIMIT 20 OFFSET 0;
```

**Назначение:**

Первые 20 самых дорогих активных товаров. Цена убывает по мере перехода к следующим страницам, строки не повторяются.

**Запрос 8. Страница 2 (OFFSET 20)**

**Текст запроса:**

```
-- Страница 2
SELECT id, title, price
FROM products
WHERE is_active
ORDER BY price DESC, id ASC
LIMIT 20 OFFSET 20;
```

**Назначение:**

Товары с 21-го по 40-й в том же порядке. Цена убывает по мере перехода к следующим страницам, строки не повторяются.

**Запрос 8. Страница 3 (OFFSET 40)**

**Текст запроса:**

```
-- Страница 3
SELECT id, title, price
FROM products
WHERE is_active
ORDER BY price DESC, id ASC
LIMIT 20 OFFSET 40;
```

**Назначение:**

Товары с 41-го по 60-й. Цена убывает по мере перехода к следующим страницам, строки не повторяются.

7. Для каждого запроса приведите его текст, назначение и фрагмент результата.

**7. Сводная таблица запросов**

Текст, назначение и фрагмент результата каждого запроса приведены в соответствующих разделах выше. Сводная информация о запросах:

| № | Назначение | Использованные конструкции |
|---|---|---|
| 1 | Уникальные пары «бренд — категория» | DISTINCT, AS, ORDER BY |
| 1a | Самый дешёвый товар в каждой категории | DISTINCT ON |
| 2 | Активные товары, дешёвые или дорогие, в двух категориях | WHERE, AND, OR, скобки |
| 3 | Товары в диапазоне цен | BETWEEN |
| 4 | Товары заданных категорий и брендов | IN |
| 5 | Поиск товаров по слову в названии | ILIKE, %, LIMIT |

| № | Назначение | Использованные конструкции |
|---|---|---|
| 5а | Поиск по шаблону с фиксированной длиной | LIKE, _ |
| 6 | Товары без описания | IS NULL |
| 6а | Подстановка текста вместо NULL | COALESCE |
| 6б | Сравнение «= NULL» и «IS NULL» | COUNT, трёхзначная логика |
| 7 | Сортировка по категории и цене | ORDER BY ... ASC, ... DESC |
| 8 | Постраничный вывод по 20 товаров | ORDER BY, LIMIT, OFFSET |

## Заключение

В ходе работы таблицы базы данных «Онлайн-зоомагазин» были наполнены согласованными тестовыми данными с помощью generate_series и random(); в таблицах товаров и заказов находится более тысячи строк. Составлен набор запросов, демонстрирующих DISTINCT, псевдонимы, комбинирование условий через AND и OR, предикаты BETWEEN, IN, LIKE/ILIKE и IS NULL, сортировку по нескольким ключам и постраничный вывод через LIMIT и OFFSET. Отдельно рассмотрена трёхзначная логика SQL: сравнение с NULL обычными операторами не работает, для проверки используется IS NULL.